КАК ТЕСТЫ ОБЪЯСНИТЬ

Методическое объяснение тестирования EH/Unwind Analyzer для комиссии

Документ подготовлен для защиты проекта: как объяснять качество, покрытие тестами, связь 16 CTest-сценариев, 27 эвристических признаков и 78 тестовых PE-бинарников.

Короткая формула для защиты
В проекте реализовано 27 диагностических признаков. Они проверяются многоуровневой системой из 16 автоматизированных CTest-сценариев и расширенным корпусом из 78 PE-бинарников. Числа не обязаны совпадать, потому что признак - это диагностическое правило, CTest-тест - это сценарий проверки кода, а бинарник - это тестовый входной артефакт.

Актуальная проверка: 13 августа 2026. CTest: 16/16 passed. Расширенный корпус: 78 PE-образцов обработаны, сгенерированы HTML/JSON/CSV/Ghidra-CSV отчеты.

1. Зачем нужен этот документ

На защите вопрос о тестировании обычно является вопросом не только о количестве тестов. Комиссия хочет понять, насколько проект контролируем, воспроизводим и устойчив. Поэтому важно объяснять тестирование не как список команд, а как систему доказательств качества. Для данного проекта качество подтверждается на нескольких уровнях: низкоуровневые unit-тесты, регрессионные PE-образцы, проверка отчетов, проверка CLI-контракта, fuzz/mutation-проверка и контроль релизной поставки.

Главная ошибка в устном объяснении - сказать: “у меня 27 признаков, но 16 тестов”. Такая фраза звучит как несоответствие. Правильнее сказать: “у меня 27 диагностических признаков, и они проверяются 16 тестовыми сценариями разного уровня; один сценарий может покрывать несколько признаков, а расширенный корпус из 78 бинарников демонстрирует работу признаков на конкретных PE-входах”.

> Как сказать комиссии
> Тестирование построено не по принципу “один признак - один тест”, а по принципу test design: отдельно проверяется парсер, отдельно эвристический слой, отдельно формат отчетов, отдельно CLI-контракт, отдельно устойчивость к поврежденным PE и отдельно демонстрационный корпус. Это более правильная структура для системного инструмента анализа бинарных файлов.

2. Три разных числа: 16, 27 и 78

Число	Что это	Как объяснять
16	CTest-сценарии	Автоматические проверки, зарегистрированные в CMake/CTest. Это тестовые программы и Python-сценарии, которые запускаются как quality gate.
27	Эвристические признаки	Диагностические правила модели анализа. Они описывают, какие аномалии инструмент умеет обнаруживать в PE, .pdata, .xdata, handler placement и комбинациях признаков.
78	Тестовые PE-бинарники	Входные образцы, специально подготовленные для демонстрации нормальных, статических, динамических и смешанных сценариев.

Эти числа относятся к разным слоям. Поэтому требование “должно быть 27 тестов, потому что 27 признаков” некорректно. Один CTest-сценарий может проверять сразу несколько признаков, а один бинарник может активировать несколько findings одновременно. В обратную сторону тоже возможно: один признак может проверяться и unit-тестом, и корпусным бинарником, и проверкой отчета.

Наиболее сильное объяснение звучит так: CTest отвечает за автоматический контроль кода; 27 признаков отвечают за функциональную полноту диагностической модели; 78 бинарников отвечают за демонстрационное и регрессионное покрытие входных PE-сценариев.

3. Что именно считается качеством для этого проекта

Для EH/Unwind Analyzer качество нельзя свести только к “программа запускается”. Инструмент анализирует бинарный формат, где ошибки чтения на несколько байтов могут полностью изменить смысл результата. Поэтому критерии качества делятся на несколько групп.

- Корректность parsing layer. Парсер должен правильно читать PE headers, sections, Exception Directory, RUNTIME_FUNCTION и UNWIND_INFO, включая CountOfCodes, дополнительные UWOP slots, padding, flags, handler tail и CHAININFO.

- Безопасность на поврежденных входах. При malformed PE инструмент не должен падать, зависать или читать за пределами буфера. Он должен либо создать частичный отчет, либо корректно вернуть код ошибки.
- Корректность эвристического слоя. Finding должен появляться только тогда, когда выполнено условие признака. Счетчики, severity и итоговая оценка должны быть согласованы между собой.
- Объяснимость отчета. JSON и HTML должны показывать не только итог, но и доказательства: RVA, секцию, file offset, affected count, причину parse failure, значения expected/available и матрицу всех 27 признаков.
- Стабильность интерфейса. CLI должен иметь предсказуемые exit codes, help/version, валидацию аргументов, атомарную запись отчетов и защиту от утечки target arguments.
- Релизная готовность. В поставке не должно быть временного мусора, несовместимых версий schema, случайных артефактов сборки и файлов, которые мешают заказчику использовать инструмент.

4. Тестовая архитектура

Тестирование построено слоями. Нижний слой проверяет атомарные функции ядра. Средний слой проверяет связку “анализатор -> отчет -> схема -> семантические инварианты”. Верхний слой проверяет пользовательский контракт CLI и релизную поставку. Отдельно стоит robustness testing: mutation_fuzz имитирует поврежденные PE и проверяет, что инструмент остается управляемым.

Уровень	Что проверяется	Почему это важно
Unit	Парсер, эвристики, матрица, JSON/HTML генерация.	Локализует ошибки в конкретном модуле и быстро показывает, что сломалось.
Integration	Запуск реального analyzer на PE-образцах, проверка JSON schema и semantic validator.	Проверяет, что модули работают вместе, а не только по отдельности.
Corpus regression	Подготовленные PE-бинарники с известными нарушениями.	Доказывает, что признаки обнаруживаются на входных файлах, близких к реальному формату.
CLI contract	Публичные флаги, ошибки аргументов, exit codes, atomic writes.	Защищает пользователя и будущий backend от непредсказуемого поведения.
Robustness / fuzz	120 мутаций входных PE.	Проверяет отказоустойчивость к поврежденным файлам.
Release hygiene	Метаданные, чистота поставки, версия schema.	Показывает готовность к передаче заказчику.

5. Полный список 16 CTest-сценариев

Ниже важно объяснять не “почему их всего 16”, а что каждый из них является тестовым сценарием. Сценарий может содержать много assert-ов, запусков инструмента, проверок файлов и проверок инвариантов.

CTest	Уровень	Что проверяет	Зачем нужен
unit_unwind	Unit	Проверяет низкоуровневый разбор x64 UNWIND_INFO: заголовок, CountOfCodes, операции UWOP, дополнительные slots, tail и chain-логику.	Доказывает, что базовый парсер не строит выводы из неправильно прочитанных байтов.
unit_heuristics	Unit	Проверяет расчет базовых статических эвристик и счетчиков finding-ов.	Нужен, чтобы 27 признаков работали как модель анализа, а не как текстовые метки.

CTest	Уровень	Что проверяет	Зачем нужен
unit_heuristics_mitigations	Unit	Проверяет контекстные смягчения и предотвращение завышенной тревожности.	Защищает от ситуации, когда любой нестандартный файл автоматически считается подозрительным.
unit_heuristics_context	Unit	Проверяет учет секций, прав доступа, handler targets, reuse и дополнительного контекста.	Показывает, что анализатор смотрит на взаимосвязи, а не только на одно поле.
unit_heuristics_combos	Unit	Проверяет комбинированные признаки: когда несколько слабых сигналов вместе дают более сильный вывод.	Это важно для защиты, потому что проект анализирует корреляции между признаками.
unit_core_edges	Unit	Проверяет граничные случаи ядра: адресные преобразования, диапазоны, ошибки маппинга и защиту от выхода за границы.	Без этого malformed PE мог бы ломать анализ или давать ложные выводы.
unit_report_json	Unit / contract	Проверяет JSON-отчет: обязательные поля, числовые счетчики, структура evidence и machine-readable контракт.	JSON - основной интерфейс для будущей БД, backend и frontend.
unit_report_html	Unit / presentation	Проверяет HTML-отчет, включая наличие матрицы 27 признаков и диагностических разделов.	Комиссии можно показать, что отчет тестируется как часть продукта, а не делается “на глаз”.
unit_matrix_report	Unit / presentation	Проверяет итоговую матрицу, статусы признаков и читаемость сводки.	Матрица помогает объяснить, что сработало, что не сработало, и почему.
regression_samples	Integration	Прогоняет минимальный набор PE-образцов, сверяет JSON schema, semantic validator и ожидаемые поля отчета.	Это быстрый контроль: после изменения кода старые гарантии не должны исчезнуть.
dynamic_validator	Integration / dynamic	Проверяет валидатор динамических отчетов для VEH, UEF, VCH и RTL function table сигналов.	Динамический слой проверяется по структуре evidence и корректности обязательных runtime-полей.
report_semantics	Integration / report	Проверяет не только схему JSON, но и смысловые инварианты: размеры функций, счетчики, секционные права, dynamic-сводку.	Схема ловит форму, semantic validator ловит противоречия внутри отчета.
release_metadata	Release	Проверяет метаданные релиза, версии, schema_version и служебную информацию.	Защищает передачу заказчику от несогласованных версий.
release_hygiene	Release	Проверяет чистоту релизной поставки: отсутствие лишних временных файлов, мусора и небезопасных артефактов.	Это тест не алгоритма, а готовности продукта к передаче.
cli_contract	CLI / integration	Проверяет help/version, ошибки аргументов, exit codes, atomic output, redaction target arguments, partial reports.	Показывает, что инструмент предсказуемо ведет себя как CLI-продукт.
mutation_fuzz	Robustness	Запускает 120 детерминированных мутаций PE-файлов: битовые инверсии, обрезки, вставки, удаление блоков, порчу e_lfanew.	Доказывает устойчивость парсера к поврежденным входам: он должен отказать корректно, а не падать.

6. Почему 16 тестов достаточно осмысленно для 27 признаков

Число CTest-тестов не равно числу проверок внутри них. Например, mutation_fuzz в CTest считается одним тестом, но внутри запускает 120 мутационных случаев. regression_samples считается одним тестом, но внутри запускает несколько PE-образцов, строит JSON-отчеты, проверяет schema, semantic validator и ожидаемые значения. unit_report_html считается одним тестом, но проверяет наличие матрицы всех 27 признаков в HTML.

Поэтому показатель качества здесь - не совпадение чисел, а наличие traceability: для каждого диагностического признака есть место, где он проверяется логикой, и есть бинарный сценарий, который демонстрирует его срабатывание.

> Готовый ответ
> Если спросят: “почему признаков 27, а тестов 16?”, отвечать так: “Потому что 27 - это количество диагностических признаков модели, а 16 - количество автоматизированных тестовых сценариев. Один сценарий покрывает несколько признаков, а отдельные сценарии проверяют не признаки, а свойства продукта: schema, CLI, устойчивость к malformed input, атомарность записи отчетов и релизную чистоту. Дополнительно есть 78 тестовых бинарников как входной корпус для демонстрации и регрессии.”

7. Расширенный корпус из 78 PE-бинарников

Расширенный корпус нужен не вместо CTest, а вместе с CTest. CTest - это автоматический quality gate разработчика. Корпус бинарников - это тестовые данные и демонстрационная база для защиты, отчета и проверки конкретных аномалий. В текущей версии корпуса: 60 static, 9 dynamic и 9 mixed образцов.

Категория	Количество	Назначение
static	60	Проверка PE layout, Exception Directory, .pdata, .xdata, handler placement, reuse/dominance и combo-признаков без запуска target.
dynamic	9	Сценарии для runtime-сигналов: VEH, UEF, VCH, RTL function table и исключения при исполнении.
mixed	9	Сценарии, где статическая странность должна сопоставляться с динамическим поведением.
Итого	78	Расширенный демонстрационный и регрессионный PE-корпус.

По корпусу были сгенерированы 78 HTML, 78 JSON, 78 CSV и 78 Ghidra-CSV отчетов. Это важно: проверяется не только факт запуска анализатора, но и способность выпускать все форматы вывода для каждого образца.

Четыре специальных образца с некорректной Exception Directory ожидаемо возвращают код частичного анализа 1. Это не провал тестирования. Это проверка правильного поведения на malformed input: инструмент не падает, а формирует частичный структурный отчет и фиксирует соответствующий finding.

8. Матрица трассировки 27 признаков

Traceability matrix показывает связь между диагностическим признаком, смыслом проверки и тестовым покрытием. Именно эту таблицу удобно использовать на защите, если комиссия спрашивает: “а где доказательство, что все признаки проверялись?”.

Код признака	Слой	Что означает	Чем покрывается
pe_section_raw_oob	PE layout	Raw-диапазон секции выходит за границы файла.	unit_core_edges, cli_contract, corpus 21-22
pe_section_virtual_oob	PE layout	Virtual-диапазон секции выходит за SizeOfImage.	unit_core_edges, corpus 23-24
pe_section_raw_overlap	PE layout	Физические диапазоны секций пересекаются.	unit_core_edges, corpus 25
pe_section_virtual_overlap	PE layout	RVA-диапазоны секций пересекаются.	unit_core_edges, corpus 26

Код признака	Слой	Что означает	Чем покрывается
pe_section_misaligned	PE layout	Нарушены alignment-поля или выравнивание секций.	cli_contract, corpus 27-28
pe_exception_directory_oob	PE layout	Exception Directory не мапится или имеет некорректный размер.	cli_contract, corpus 29-32
no_runtime_functions	.pdata	В PE x64 нет читаемых RUNTIME_FUNCTION записей.	heuristics, corpus 33
invalid_ranges	.pdata	BeginAddress >= EndAddress.	unit_heuristics, corpus 04, 34-35
rf_oob	.pdata	Диапазон функции выходит за границы image.	unit_heuristics, corpus 05, 36-37
unwind_rva_oob	.pdata -> .xdata	UnwindInfoAddress не указывает на допустимые байты.	regression_samples, corpus 03, 19-20, 38-39
rf_unsorted	.pdata	Таблица RUNTIME_FUNCTION не отсортирована по BeginAddress.	unit_heuristics, corpus 06, 40, 58
rf_overlaps	.pdata	Диапазоны функций пересекаются.	unit_heuristics, corpus 06, 41, 74
duplicate_begin	.pdata	Несколько записей имеют одинаковый BeginAddress.	unit_heuristics, corpus 06, 42
unwind_parse_fail	.xdata	UNWIND_INFO не может быть надежно разобран.	regression_samples, corpus 03, 38-44, 77
unwind_anomalies	.xdata	Формально прочитанная metadata содержит внутренние аномалии.	unit_unwind, corpus 07-08, 45-50, 78
unwind_reuse_strong	Reuse	Сильное повторное использование одного UnwindInfoRVA.	unit_heuristics_combos, corpus 14, 59
unwind_reuse_moderate	Reuse	Умеренное повторное использование UnwindInfoRVA.	unit_heuristics_context, corpus 58
handler_dominance	Handler	Один handler доминирует над большим числом функций.	unit_heuristics_context, corpus 14, 60-61
unwind_in_exec_section	Placement	UNWIND_INFO размещен в исполняемой секции.	unit_heuristics_context, corpus 10, 54, 70
unwind_unusual_section	Placement	UNWIND_INFO находится в нетипичной секции.	unit_heuristics_context, corpus 09, 71
unwind_writable_section	Placement	UNWIND_INFO находится в writable-секции.	regression_samples, corpus 11-12, 69
handler_nonexec_section	Handler placement	Handler RVA ведет в неисполняемую область.	unit_heuristics_combos, corpus 10, 51-52, 60, 72
handler_writable_section	Handler placement	Handler target находится в writable-секции.	unit_heuristics_context, corpus 12, 53
placement_exec_nonexec_handler_combo	Combo	Комбинация executable metadata и non-executable handler.	unit_heuristics_combos, corpus 10
placement_integrity_combo	Combo	Связка проблем размещения и целостности metadata.	unit_heuristics_combos, corpus 13
reuse_dominance_combo	Combo	Связка reuse и handler dominance.	unit_heuristics_combos, corpus 14
writable_metadata_handler_combo	Combo	Связка writable metadata и writable handler.	unit_heuristics_combos, corpus 12

9. Как работает regression_samples

Тест regression_samples - это один CTest-тест, но внутри он выполняет полноценный end-to-end контроль. Он берет компактный набор seed PE из tests/samples, при необходимости пересобирает

отсутствующие samples, запускает analyzer в режимах --dump и --heuristics --json, затем проверяет JSON report.schema.json, semantic validator и ожидаемые значения из manifest.

Смысл этого теста - не заменить все 78 демонстрационных бинарников, а дать быстрый регрессионный gate. Он нужен при каждой правке кода: если разработчик случайно изменил scoring, сломал JSON, нарушил semantic invariant или перестал находить базовую аномалию, regression_samples должен это обнаружить.

> Как объяснить разницу между 6 regression samples и 78 бинарниками
> Внутренний CTest regression_samples использует небольшой быстрый набор для постоянного контроля сборки. Расширенный набор из 78 PE - это более широкий демонстрационный и регрессионный корпус, по которому дополнительно генерируются отчеты. Это нормальная практика: быстрый обязательный gate плюс расширенный корпус для покрытия сценариев и презентации.

10. Проверка отчетов: почему это отдельная часть качества

Для данного проекта отчет - не вторичный красивый файл, а один из основных результатов анализа. Если analyzer правильно нашел проблему, но отчет не показывает поле, RVA, секцию или причину, ценность для reverse engineer-а падает. Поэтому тестирование отчетов вынесено в отдельные сценарии: unit_report_json, unit_report_html, unit_matrix_report, report_semantics.

JSON проверяется как машинный контракт: будущий backend, база данных или frontend должны получать стабильную структуру. HTML проверяется как экспертный отчет: человек должен видеть summary, ключевые диагностические ошибки, coverage статических проверок, матрицу 27 признаков и таблицу RUNTIME_FUNCTION. Semantic validator проверяет, что отчет не только формально валиден, но и внутренне непротиворечив.

Проверка	Что ловит
JSON schema	Отсутствие обязательных полей, неправильные типы, нарушение версии schema.
Semantic validator	Несогласованные счетчики, неправильный size функции, противоречия между section flags и derived fields.
HTML unit test	Отсутствие ключевых разделов, ошибки escaping, отсутствие матрицы 27 признаков.
Matrix report test	Неправильное отображение активных/неактивных признаков и итоговой матрицы.

11. CLI contract как тест продукта

Комиссия может спросить: “это тесты алгоритма или продукта?”. Важно ответить: есть и то, и другое. cli_contract проверяет именно продуктовый слой: как инструмент ведет себя для пользователя и для будущей интеграции с backend.

- Проверяются --help и --version, включая строку Report schema: 1.2.0.
- Проверяются invalid arguments: неизвестная опция, неправильный --limit, --max-chain-depth, --timeout-ms, --trace-steps.
- Проверяются exit codes: 0 для успешного анализа, 1 для частичного структурного отчета на malformed input, 2 для ошибки использования CLI.
- Проверяется atomic output: после записи отчетов не должны оставаться временные .tmp-файлы.

- Проверяется redaction target arguments: потенциально чувствительные аргументы target-процесса не должны попадать в metadata отчета.
- Проверяется создание JSON, обычного CSV и Ghidra CSV.

12. Mutation fuzz: зачем он нужен

Mutation fuzz - это проверка устойчивости, а не проверка “нашел конкретный признак”. Скрипт берет seed PE-образцы и делает 120 детерминированных повреждений: инвертирует случайные биты, затирает фрагменты, обрезает файл, вставляет случайные байты, удаляет диапазоны, портит e_lfanew и ключевые позиции файла.

Ожидаемое поведение: analyzer может принять мутированный файл и создать JSON, либо корректно отвергнуть его кодом 1. Недопустимое поведение: crash, timeout, invalid JSON после успешного кода, нарушение schema_version или чтение за пределами буфера.

> Как сказать комиссии
> Fuzz-тестирование показывает, что парсер не рассчитан только на “идеальные учебные файлы”. Он устойчив к поврежденным входам и ведет себя контролируемо: либо строит отчет, либо корректно отказывает без падения процесса.

13. Динамический слой и его тестирование

Динамическая часть проекта относится к Windows runtime: debug events, VEH, VCH, UEF, exceptions, handler hits, RtlAddFunctionTable и RtlInstallFunctionTableCallback. На macOS или Linux полноценный dynamic run Windows-процесса невозможен без Windows-среды, поэтому динамический слой разделяется на две части: проверка структуры/валидатора dynamic reports и практический прогон на Windows.

Тест dynamic_validator проверяет, что валидатор отличает корректный dynamic evidence от некорректного: VEH registration/hit, UEF set/hit, VCH registration/hit, RTL function table calls. Отдельно проверяется negative case: если report timed_out, валидатор не должен принимать его как нормальный успешный динамический отчет.

Если комиссия спрашивает, почему dynamic на macOS не подтверждает runtime, честный ответ: dynamic layer технически привязан к Windows debugging/runtime API. На macOS можно проверить static, формат отчетов и валидатор dynamic evidence, но фактические handler hits и runtime exception routes должны проверяться на Windows. Это не дефект архитектуры, а природа предметной области.

14. Что означают exit codes при тестировании

Код	Смысл	Как объяснять
0	Анализ выполнен успешно.	Файл прочитан, отчет создан, критической ошибки использования CLI нет.
1	Частичный анализ или контролируемая ошибка анализа входного файла.	Например, Exception Directory malformed. Это ожидаемо для специально сломанных PE: инструмент должен создать частичный отчет и finding.
2	Ошибка использования CLI.	Нет входного файла, неизвестная опция, недопустимое значение параметра.

На защите важно не говорить “четыре бинарника упали”. Они не упали. Они вернули ожидаемый код 1, потому что были специально повреждены на уровне Exception Directory. Разница

принципиальная: падение - это crash или неконтролируемое завершение, а код 1 с частичным отчетом - это заданное поведение инструмента.

15. Методика тестировщика: как это описывать научно

Работу над тестированием можно описать как построение test basis, test design и evidence collection. Test basis - это требования к анализатору: корректно читать PE x64 EH/Unwind metadata, выявлять структурные аномалии, формировать отчеты и устойчиво обрабатывать malformed input. Test design - это разбиение требований на уровни проверки: unit, integration, corpus, CLI, fuzz, release. Evidence collection - это сохраненные результаты: CTest output, manifest_extended.json, сгенерированные отчеты, schema validation и semantic validation.

Этап	Что делается	Результат
1. Требование	Формулируется проверяемое свойство: например, CountOfCodes должен покрывать все UWOP operand slots.	Появляется критерий ожидаемого поведения.
2. Тестовый вход	Создается PE, где UWOP_ALLOC_LARGE требует 3 slots, а CountOfCodes равен 1.	Появляется бинарник 77_static_diag_uwop_slots_beyond_count.exe.
3. Запуск	Analyzer запускается с --heuristics и выводит JSON/HTML/CSV.	Получается машинный и человекочитаемый отчет.
4. Assert	Проверяется finding unwind_parse_fail, diagnostic field, expected_slots/available_slots, отсутствие crash.	Фиксируется корректность реакции.
5. Объяснение	В HTML поднимается блок “Ключевые диагностические ошибки”.	Комиссия видит не магический итог, а доказательную цепочку.

16. Что тестирование доказывает, а что не доказывает

Хорошая защита не должна преувеличивать. Тесты доказывают корректность реализации на заданной модели требований и на подготовленных классах входов. Они не доказывают математически, что любой PE-файл в мире будет классифицирован идеально. Для бинарного анализа это нормальная инженерная позиция.

Доказывается	Не доказывается
Парсер обрабатывает нормальные и специально поврежденные PE без неконтролируемого падения в проверенных сценариях.	Абсолютная корректность на всех возможных PE, packer-ах, obfuscator-ах и compiler output.
27 признаков присутствуют в модели отчета и имеют тестовое/корпусное покрытие.	Что этих 27 признаков достаточно для любой вредоносной техники.
JSON/HTML/CSV/Ghidra-CSV формируются и проверяются автоматикой.	Что каждый внешний пользователь интерпретирует отчет без предметных знаний.
Dynamic evidence валидируется структурно, а фактический runtime-прогон должен выполняться на Windows.	Что dynamic layer полноценно исполняется на macOS/Linux без Windows runtime.

Честный ответ про точность

Это не ML-модель и не антивирусный классификатор. Поэтому “точность модели” нельзя честно заявлять как accuracy на массовом датасете без размеченного реального корпуса. В проекте корректнее говорить о функциональной точности на подготовленном регрессионном корпусе: ожидаемые признаки для тестовых PE совпадают с полученными отчетами, а устойчивость проверяется fuzz/mutation-тестированием.

17. Как отвечать на типовые вопросы комиссии

Почему 16 тестов, если признаков 27?

Потому что тест - это сценарий проверки, а признак - это диагностическое правило. Один сценарий может проверять несколько признаков, а часть тестов проверяет не признаки, а свойства продукта: CLI, JSON schema, HTML, release hygiene, fuzz robustness.

Почему не сделать один тест на каждый finding?

Можно, но это не единственный и не всегда лучший дизайн. В системном анализаторе важнее покрыть слои: parser, heuristics, reports, CLI, robustness. При этом traceability matrix показывает, где каждый finding проверяется.

Что доказывают 78 бинарников?

Они являются controlled test corpus: каждый PE создан или выбран для демонстрации конкретного нормального, статического, динамического или смешанного сценария. По ним сгенерированы отчеты, поэтому можно показать evidence для каждого класса аномалий.

Почему regression_samples внутри CTest использует меньше образцов, чем расширенный корпус?

Потому что это быстрый gate для каждой сборки. Расширенный корпус используется как более тяжелый демонстрационный и регрессионный набор. Это обычная практика: быстрые обязательные тесты плюс расширенные проверки.

Что будет, если файл сильно поврежден?

Инструмент должен либо сформировать частичный отчет с finding, либо завершиться контролируемым кодом. mutation_fuzz проверяет именно это: отсутствие crash, timeout и invalid successful report.

Как проверяется отчет?

Через JSON schema, semantic validator и unit-тесты HTML/матрицы. Отчет считается частью продукта, потому что reverse engineer принимает решение по evidence, а не только по exit code.

Динамика проверена?

Динамический слой зависит от Windows runtime/debugging API. На macOS проверяются static, report contracts и dynamic report validator. Фактические handler hits и runtime routes должны проверяться в Windows-среде.

Почему некоторые malformed binaries возвращают code 1?

Это ожидаемый partial-analysis code. Файл специально поврежден так, что Exception Directory не может быть корректно прочитана. Инструмент не падает, а фиксирует проблему и создает частичный отчет.

Можно ли по тестам сказать, что проект готов?

Можно сказать, что проект прошел текущий автоматизированный quality gate: 16/16 CTest, расширенный корпус обработан, отчеты сгенерированы. Но промышленное утверждение о точности на рынке потребовало бы отдельный размеченный real-world corpus.

Что является главным доказательством качества?

Не одно число, а совокупность: unit-тесты парсера, тесты эвристик, regression samples, schema/semantic validation, CLI contract, fuzz на 120 мутаций, 78 PE-образцов и проверенные форматы отчетов.

18. Готовый устный текст на 1-2 минуты

“Тестирование проекта построено многоуровнево. Я не считал качество только количеством тестов, потому что в таком инструменте есть разные сущности: 27 эвристических признаков, 16 автоматизированных CTest-сценариев и 78 тестовых PE-бинарников. 27 признаков - это диагностическая модель анализа. 16 CTest-сценариев - это автоматический gate, который проверяет парсер, эвристики, JSON и HTML отчеты, CLI-контракт, semantic validator, динамический валидатор, релизную чистоту и устойчивость через mutation fuzz. 78 бинарников - это расширенный controlled corpus, где каждый файл демонстрирует отдельный нормальный или поврежденный сценарий. По ним сгенерированы отчеты во всех форматах: HTML, JSON, CSV и Ghidra CSV.”

“Поэтому числа 16, 27 и 78 не должны совпадать. Они относятся к разным уровням качества. Один CTest-сценарий может проверять несколько признаков, а один бинарник может активировать несколько findings. Чтобы показать связь, в проекте есть матрица 27 признаков и расширенный manifest, где описано, что сломано в каждом PE. Последний прогон показал 16/16 passed, а расширенный корпус из 78 PE был обработан и по нему сгенерированы отчеты.”

19. Практический чек-лист перед защитой

- Показать CTest output: 16/16 passed.
- Показать папку с 78 тестовыми бинарниками: static/dynamic/mixed.
- Открыть HTML-отчет с матрицей 27 признаков.
- Открыть диагностический пример 77_static_diag_uwop_slots_beyond_count.html и показать expected_slots=3; available_slots=1.
- Объяснить, что code 1 для malformed Exception Directory - это не crash, а ожидаемый partial-analysis behavior.
- Подчеркнуть, что JSON является контрактом для будущей БД/backend/frontend.
- Отдельно сказать, что dynamic runtime требует Windows-среду, потому что использует Windows exception/debugging semantics.

20. Итог

Тестирование проекта следует объяснять как систему контроля качества, а не как набор случайных запусков. На нижнем уровне проверяются parsing и эвристики. На среднем - отчеты, schema, semantic consistency и регрессионные PE. На верхнем - CLI, релизная поставка и robustness. Расширенный корпус из 78 бинарников нужен для демонстрации покрытия признаков и для воспроизводимого анализа. 16 CTest-сценариев являются автоматическим quality gate, а 27 признаков - функциональной моделью анализа. Именно такое разделение делает объяснение профессиональным и защищаемым.